

ADR - NPI

Até SPRINT 2

ADR 001: Arquitetura de Monolito Modular

Status: Aceito

Data: 30/04/2026

Contexto: Precisávamos decidir a estrutura organizacional do sistema CliniDesk. Havia duas opções: Microserviços ou Monolito.

Decisão: Optamos por uma Arquitetura de Monolito, onde o Backend (Node/Express) gerencia todas as regras de negócio em um único serviço, e o Frontend (React) consome essa API.

Justificativa:

1. **Redução de Complexidade:** A comunicação entre múltiplos serviços (microserviços) aumentaria drasticamente o tempo de entrega.
2. **Custo de Infraestrutura:** Um monolito é mais barato e simples de hospedar para um MVP de clínica local.
3. **Consistência de Dados:** Facilita transações atômicas e garante que os dados dos pacientes e usuários estejam sempre sincronizados.

Consequência:

(Positivo)

Escalabilidade de Módulos Críticos: Módulos como o de "Agendamento" ou "Prontuário Eletrônico" podem ser escalados de forma independente caso o CliniDesk sofra um pico de acessos em horários comerciais.

ADR 002: Uso de Banco de Dados NoSQL (MongoDB)

Status: Aceito

Data: 30/04/2026

Contexto: O sistema precisa armazenar agendamentos que podem sofrer alterações frequentes de estrutura (novos campos de observação, tipos de convênio, etc).

Decisão: Utilizar o **MongoDB** como banco de dados principal.

Justificativa:

1. **Esquema Flexível (Schema-less):** Como o projeto é um MVP para o NPI, os requisitos mudam rápido. O MongoDB permite adicionar campos sem a necessidade de "migrations"

complexas como no SQL.

2. **Produtividade da Equipe:** O formato de documentos (BSON) é muito similar ao JSON utilizado no JavaScript, diminuindo a curva de aprendizado.

3. **Integração Nativa:** A stack Node + Mongoose + MongoDB é extremamente estável e bem documentada.

Consequência:

(Positiva)

Conformidade ACID: Essencial para o contexto clínico, garantindo que atualizações em prontuários ou registros de consultas sejam processadas de forma segura, sem perda de dados em caso de falhas.

Consultas Complexas (Relatórios): Facilita a geração de relatórios gerenciais e estatísticos (ex: "quantas consultas o médico X realizou no mês Y") usando JOINS de forma performática.

(negativo)

Esquema Rígido: Qualquer alteração na estrutura de dados (ex: adicionar um novo campo no Prontuário) exige a execução de uma Migration e pode causar downtime se não for bem planejada.

ADR 003: Autenticação via JSON Web Token (JWT) com Login por Username

Status: Aceito

Data: 30/04/2026

Contexto: Necessidade de identificar usuários na recepção de forma rápida, sem exigir e-mails longos, e manter a sessão segura.

Decisão: Implementar autenticação **Stateless via JWT**, substituindo o campo email por username (iniciais/login curto).

Justificativa:

1. **Agilidade Operacional:** A recepcionista precisa de velocidade. Digitar erlon é mais rápido que erlon.silva@clinica.com.br.

2. **Escalabilidade (Stateless):** O servidor não precisa guardar estados de sessão na memória. O token carregado pelo cliente (Albert) contém toda a informação necessária para autorizar as rotas.

3. **Segurança:** O JWT permite expiração automática e assinatura digital, protegendo os dados sensíveis dos pacientes.

(Positivo)

Segurança no Tráfego de Dados: Permite que informações do profissional de saúde (como ID e nível de acesso) sejam transportadas de forma compacta e assinada digitalmente.

Performance: Reduz a necessidade de consultas constantes ao banco de dados apenas para verificar se o usuário ainda está logado, já que o token é autossuficiente.

(negativo)

Complexidade de Refresh: Exige a implementação de uma lógica de Refresh Token para que o usuário não precise fazer login novamente a cada 15 minutos.

ADR 004: Padronização de Configurações via Variáveis de Ambiente (.env)

Status: Aceito

Data: 30/04/2026

Contexto: O projeto estava enfrentando erros de conexão (undefined) e riscos de segurança ao expor senhas do banco de dados no código-fonte.

Decisão: Adotar o uso obrigatório de arquivos **.env** e da biblioteca **dotenv** em todos os ambientes de desenvolvimento.

Justificativa:

1. **Segurança (DevSecOps):** Impede que credenciais críticas (como a **MONGO_URI**) sejam enviadas para o GitHub.
2. **Portabilidade:** Facilita a troca de ambiente (Desenvolvimento vs. Produção) sem precisar alterar uma única linha de código lógico.
3. **Padronização da Equipe:** Todos os desenvolvedores seguem o mesmo modelo de configuração (**.env.example**), evitando o erro de "na minha máquina funciona".

(Positivo)

Padronização da Equipe: Cria um padrão claro para os desenvolvedores: "onde cada arquivo deve ficar", reduzindo a confusão durante o desenvolvimento do projeto.

(Negativo)

Curva de Aprendizado: Exige que todos os membros da equipe entendam os limites de cada camada, evitando que regras de negócio "vazem" para dentro dos controladores ou do banco de dados.